

CartPilot — A Multi-Agent E-Commerce Decision and Operations Management System

[Student / team names] • [Institution] • [Department] • [Project guide]

Submission date: [Insert required date]

Prepared from verified project evidence dated 3 October 2026. Academic identity and institution-specific formatting require completion.

Contents

Abstract

1. Introduction
 2. Problem statement
 3. Objectives
 4. Technical background
 5. Proposed system and methodology
 6. Architecture
 7. Specialist algorithms
 8. Goal-based orchestration
 9. Database and API design
 10. Implementation
 11. Approval, execution and security
 12. Testing and verification
 13. Demonstration and results
 14. Deployment and operations
 15. Limitations
 16. Future scope
 17. Conclusion
 18. References
- Appendix A: Algorithms
- Appendix B: API, database and security
- Appendix C: Demonstration screenshots

Abstract

E-commerce operations require coordinated decisions about product pricing, inventory replenishment, promotions and catalog quality. When these decisions are made independently, a discount may increase demand for a product already facing stockout risk, while poor listing information may be mistaken for a pricing problem. CartPilot addresses this coordination problem through a modular, recommendation-first decision-support system. Four deterministic specialist agents analyze shared merchant data and produce structured, explainable recommendations. A Master Orchestrator combines their outputs according to merchant goals, identifies conflicts, synergies and dependencies, and constructs a bounded priority plan. Proposed operations pass through typed policy validation, human approval, separate execution confirmation and current-state revalidation. Price and listing changes apply only to local data; replenishment and promotion execution are simulated and audited. A read-only Shopify integration imports store data without modifying the external store. The implementation uses a React and TypeScript frontend, a FastAPI backend and relational persistence with versioned migrations. Verification includes unit, integration, failure, workflow and browser checks. The contribution is an explainable coordination and guarded execution design rather than a trained forecasting model. Current limitations include simulated external execution, absent production authentication and no autonomous learning. Measured revenue improvements are not claimed.

1. Introduction

CartPilot is a decision-support layer above e-commerce inventory and catalog data. It combines pricing, replenishment, promotion and listing analysis in a single merchant-scoped interface. The prototype demonstrates explainable coordination and human-controlled changes using deterministic specialist modules. It does not replace a storefront or predict business performance.

2. Problem statement

Pricing, inventory, discounts and catalog quality are interdependent. A promotion may stimulate demand while stock is constrained; a weak listing may be treated as a price problem. Independent recommendations require reconciliation before acting. CartPilot addresses this coordination problem through shared context, explicit dependencies, policy validation and an auditable approval workflow. These are motivating scenarios; the project did not quantify business losses or revenue uplift.

3. Objectives

The objectives are to centralize scoped product, order, inventory and price-history signals; produce four typed, explainable specialist recommendations; coordinate goals and conflicting opportunities; require human approval and fresh validation; support bounded read-only Shopify synchronization; and provide reproducible tests and a reliable academic demonstration.

4. Technical background

The implementation uses typed HTTP APIs, relational persistence, deterministic decision rules and transactional workflow control. FastAPI and Pydantic support request/response validation. SQLAlchemy provides asynchronous persistence, while Alembic versions the schema. React components present merchant-scoped data and recommendation cards. The cited primary documentation explains these technologies. This report does not claim a peer-reviewed literature survey or a competitive benchmark.

5. Proposed system and methodology

Sense reads product, sales, available-stock and listing facts. Decide applies four specialists and a goal-based Master Orchestrator. Act validates a typed proposal, obtains approval, asks for separate execution confirmation, rechecks current state and records the result. Learn currently means retaining outcomes for human review. No trained model, automatic retraining, reinforcement learning or LLM runtime is implemented.

6. Architecture

The browser calls a FastAPI backend. Shared merchant-scoped signals feed Pricing, Restock, Promotion and Listing modules. The Master Orchestrator returns an advisory priority plan with conflicts, synergies, dependencies, blocked candidates and failures. An independent action service controls local execution. PostgreSQL is the deployment target; isolated tests and the offline demo use SQLite. Redis is optional diagnostics. Shopify import is read-only and bounded; local changes are never written back.

7. Specialist algorithms

Sales velocity is eligible sold units divided by lookback days. Available stock is $\max(0, \text{physical} - \text{reserved} - \text{unavailable})$. Pricing uses ordered demand/stock rules, Decimal rounding, movement caps and a cost-based markup floor. Restock uses lead-time demand, ceiling-rounded safety stock, buffer and an order cap. Promotion bounds discounts by inventory safety and retained gross margin. Listing quality uses weighted factual text checks; missing attributes require verification. Exact thresholds and rounding rules appear in Appendix A.

8. Goal-based orchestration

The orchestrator supports six controlled merchant goals, analyzes at most 100 active products and builds a bounded candidate queue. Balanced Growth weights inventory 0.30, revenue 0.30, margin 0.20 and catalog 0.20. Critical inventory takes precedence. Conflicts include competing price changes and promotion on constrained stock. Dependencies can require listing review before discounting. Confidence, risk and priority are distinct heuristic signals; they are not calibrated probabilities.

9. Database and API design

Fourteen modeled entities cover merchants, products, inventory, orders/items, price history, agent runs, recommendations, actions, approvals, audit logs and Shopify connection/external mappings. Foreign keys and unique identities support integrity and duplicate prevention. Money uses Decimal/Numeric(10,2), and API timestamps normalize to UTC. Three Alembic revisions end at `11shopify_read_only`. AgentRun exists as a model; the Agent Activity UI currently records browser-session activity. Appendix B lists actual APIs and entity responsibilities.

10. Implementation

The verified stack uses Python 3.12.7, FastAPI 0.142.2, Pydantic 2.13.5, SQLAlchemy 2.1.1 and Alembic 1.20.0. The frontend uses React 18.3.1, TypeScript 5.9.3, Vite 5.4.21 and Tailwind 3.4.19. PostgreSQL 16 is the container target; isolated PostgreSQL 17.11 execution was checked. These are recorded verification versions, not claims about the latest releases. Nine frontend routes cover catalog, analysis, review, audit and integrations.

11. Approval, execution and security

Analysis alone changes no product values. Action creation validates immutable typed payloads and policies. Human approval and execution confirmation are separate steps. Current price, cost, stock and source facts are checked again before execution. Transactions, atomic claims, workflow identities and audits protect retries. Price/listing execution changes local data; promotion/restock execution is simulated. There is no authenticated merchant identity. Production mode blocks action writes and Shopify routes; trusted local/private access is required.

12. Testing and verification

The recorded final regression passed 560 backend tests and 76 frontend tests across nine files. TypeScript and Vite build passed. Fresh SQLite migrations, seed/reseed, stale-state checks, nine direct browser routes, responsive layouts and the approval workflow were verified. Basic PostgreSQL migrations and local execution were checked in Phase 13. Shopify transport was mocked. Test counts are evidence of exercised behavior, not prediction accuracy. Docker/Nginx execution, cloud hosting, remote CI and locking stress remain unverified.

13. Demonstration and results

The Four-product Demo gives the Wireless Mouse eight available units and five units/day velocity. Coverage is 1.6 days and the restock suggestion is 67 units. Pricing proposes 999.00 → 1048.95. A declining-sales Speaker supports a 10% guarded discount to 1349.10; incomplete Headphones content requires listing review. Balanced Growth prioritizes Mouse restock, Headphones listing, Speaker promotion and Mouse price review. Separate confirmation applies the local Mouse price change and records `execution_completed`. A 50% proposal fails the 10% movement policy. No measured sales uplift is claimed.

14. Deployment and operations

The deployment topology is an HTTPS static frontend, a protected Python service and managed PostgreSQL. Migrations run once before serving; seeding is explicit and only for disposable demo data. `VITE_API_URL` is a public build-time value. CORS must name exact frontend origins. Production mode preserves write restrictions. A private full-workflow demo requires provider access controls. No hosting provider, cloud account or public URL has been configured for this delivery; the local demo is independent of those services.

15. Limitations

The system uses fixed heuristic rules and demand assumptions. External restock/promotion execution is simulated, Shopify writes are absent and live-store verification has not been performed. Authentication, supplier orders, autonomous learning, scientific forecasting evaluation and measured business impact are outside the implemented scope. Catalog analysis is bounded, some aggregates scale by product count, and PostgreSQL concurrency stress remains unverified.

16. Future scope

Potential extensions include authenticated merchant identity, ingress controls, measured forecasting, verified supplier adapters, carefully authorized external writes, additional commerce integrations and outcome evaluation. These require separate development and validation; they are not existing capabilities.

17. Conclusion

CartPilot demonstrates modular, explainable e-commerce coordination with a human-controlled execution boundary. The prototype presents structured priorities, exposes conflicting proposals, blocks unsafe changes and retains audit evidence. It is suitable for a trusted academic demonstration. Production expansion requires the identity, infrastructure and external-execution work identified above.

18. References

FastAPI documentation — <https://fastapi.tiangolo.com/>

SQLAlchemy asyncio — <https://docs.sqlalchemy.org/en/20/orm/extensions/asyncio.html>

Alembic tutorial — <https://alembic.sqlalchemy.org/en/latest/tutorial.html>

React documentation — <https://react.dev/>

Shopify Admin GraphQL — <https://shopify.dev/docs/api/admin-graphql/latest>

Project evidence: TEST_SUMMARY.md, FINAL_STATUS.md, phases/PHASE_13.md, phases/PHASE_14.md and docs/examples/submission-browser.json. Technology references support implementation concepts; project claims derive from local source and recorded verification.

Appendix A: Source-matched algorithms

Verified algorithms and formulas

These are deterministic business rules, not trained forecasting models. Source of truth: backend/app/services/{pricing,restock,promotion,listing,orchestration,policies}. Confidence, risk and priority are separate heuristic outputs; none predicts revenue lift.

Shared signals

Sales velocity v = eligible units sold / configured lookback days (default 14).
Orders are merchant/product scoped, within the observation window, excluding cancelled orders. A short observed history is not silently substituted as the denominator.
Available stock A = max(0, physical – reserved – unavailable); unavailable stock is important for imported Shopify inventory. Missing inventory is unknown, not evidence of a safe stock level. Coverage A/v ; zero velocity yields no finite stockout estimate.

Strong sales: $v \geq 1$ unit/day or lookback units ≥ 10 . Weak sales: sufficient data and ($v < .25$ or units ≤ 2). Low stock: $A \leq$ reorder point. High inventory: A exceeds reorder_point + $1.5 \times$ reorder_quantity or coverage exceeds 45 days. These overlap in some cases; agent decision ordering and orchestrator safety resolve competing signals.

Pricing Agent

Decision order: unknown inventory or insufficient history → hold; low stock with strong sales → bounded increase; low stock otherwise → hold; healthy stock with strong sales → increase; high stock with weak sales → decrease; otherwise hold. Default movement is 5%, capped by default 10% increase/decrease.

$\text{delta\%} = (\text{recommended} - \text{current}) / \text{current} \times 100$.
Upper cap is $\text{current} \times (1 + \text{maximum_increase}/100)$, rounded down to cents; lower cap rounds $\text{current} \times (1 - \text{maximum_decrease}/100)$ up to cents. Cost floor is $\text{cost} \times (1 + \text{MIN_MARGIN_PERCENT}/100)$, rounded up. This setting is markup over cost, not retained gross margin. Conflicting cost floor and movement limits raise a safe failure instead of silently selecting an unsafe price. Unknown cost blocks pricing.

Confidence: .20 if insufficient data; otherwise .35 + order-volume contribution (.25 for ≥ 15 recent orders, .15 for ≥ 5) + .15 if physical inventory > 0 + .10 if price history exists + .15 if $v \geq .5$, bounded to [.10,1]. Risk is high for insufficient data, cost clamp, low stock or gross margin $< 5\%$; otherwise medium for capped movement or absolute $\text{delta} \geq 5\%$; otherwise low. A high-risk pricing proposal can remain a medium priority review: urgency and execution risk are different concepts.

Example, actual four-product fixture: Mouse 999.00 → 1048.95 (+5%), high risk because stock is constrained. This is a recommendation; the product changes only after approval and separately confirmed execution.

Restock Agent

Lead-time demand $D = v \times \text{lead_time_days}$ (default 5).
Safety stock $S = \text{ceil}(v \times \text{safety_stock_days})$ (default 2).
Trigger when $v > 0$ and ($A \leq \text{reorder_point}$ or $A < D + S$).

Reorder quantity $Q = \min(\text{max_quantity}, \max(0, \text{ceil}(D+S+\text{reorder_quantity}-A)))$.
 The stored `reorder_quantity` is an additional buffer, not a supplier minimum.
 Stockout risk when $v > 0$ and $A \leq D$; equality is risky because stock is exhausted on arrival.
 Days remaining is A/v , rounded to 2 decimals, or null at zero velocity.
 Inactive/insufficient-history/zero-demand products do not receive a positive reorder.
 A capped order can leave residual exposure. No purchase order is sent or stock added.

Confidence: .20 for insufficient/zero-demand/inactive cases; otherwise .40 + .15 for known inventory + .30/.20/.10 for $\geq 15/\geq 5$ /fewer recent orders. Risk high for stockout or capped quantity, medium for a noncritical trigger, otherwise low.
 Actual Mouse fixture recommends 67 units under default policy.

Promotion Agent

Compare velocities in newer and older halves of the lookback window.
 Trend change $(\text{recent_velocity} - \text{previous_velocity})/\text{previous_velocity} \times 100$;
`previous=0` means increasing if `recent>0`, stable otherwise. Default decline threshold is strictly below -20%. Incomplete history is explicitly insufficient.
 High inventory: coverage > 45 days, or zero-sales stock ≥ 30 . Weak movement: $v < 1.5$ or declining trend. Low stock/reorder point and fresh stockout assessment block promotions.

Desired discount: 15% at zero sales, 10% for decline, otherwise 5% (configurable).
 Retained gross margin $\text{GM\%} = (\text{selling} - \text{cost})/\text{selling} \times 100$.
 Minimum safe price $F = \text{cost} / (1 - \text{minimum_margin}/100)$.
 Maximum margin-safe discount $\max(0, (1 - F/\text{current_price}) \times 100)$.
 Final discount is $\min(\text{desired}, \text{configured maximum}, \text{margin-safe maximum})$, rounded down at .01%; candidate price is rounded to cents and rechecked against F . Unsafe/no-op candidates return no promotion. Unknown cost blocks margin-dependent analysis.
 Example: fixture Speaker 1499.00 → 1349.10 (10%); orchestrator defers the competing base-price decrease. Launching a campaign remains simulated, not an external action.

Listing Agent

Clean HTML, ignore script/style text, inspect title/description lengths and duplication, category alignment and mentioned category-specific attributes. Mention does not prove an attribute is true or applicable. Unknown categories require manual review.
 Quality = $.25 \times \text{title_pass_fraction} + .35 \times \text{description_pass_fraction} + .25 \times \text{attribute_fraction} + .15 \times \text{category_alignment}$, rounded to 2 decimals.
 Title has 3 checks, description 4; unknown attribute categories use .5. Alignment is 1 for consistent, .5 for unverified, 0 otherwise. Default poor-quality threshold .6.
 Confidence .20 for sparse sources; otherwise $\min(.90, .45 + .20 \text{ if description} + .15 \text{ if category} + .10 \text{ if SKU})$. Risk high for sparse/inconsistent sources, medium for substantial changes or poor quality, otherwise low.
 Suggestions copy existing facts/identifiers, preserve negations, and request verification for missing information. No unsupported battery, material, connectivity or performance claim is invented. Keywords are extracted tokens, not measured SEO ranking predictions.

Master Orchestrator and cross-agent intelligence

Fresh merchant context → goal-based specialist selection → structured result collection
→ safety overrides → conflicts/synergies/dependencies → sorted bounded advisory queue.
Balanced Growth requests all four agents. Inventory exposure can override other goals.

Weights for Balanced Growth: inventory .30, revenue .30, margin .20, catalog .20.
score = agent_goal_weight × clamp(signal_strength,0,1) × confidence, rounded to 4 decimals.
Agent weight: restock inventory; pricing revenue+margin; promotion revenue; listing
catalog plus .5×revenue for strong listing issues. Critical inventory overrides scoring;
otherwise high at score≥.15 or severe listing issue, medium≥.05, low below .05.
Balanced Growth price-change reviews are medium. Sorting: priority, descending score,
product ID, agent name. Default action queue budget 5; omitted and blocked work remain visible.

Conflicts: base-price change vs temporary discount; demand stimulation vs constrained
inventory. Synergy: bounded price increase + replenishment; factual listing improvement
supports promotion review. Dependency: poor listing should be reviewed before discount
or price-reduction remedies. Simulated restock does not satisfy actual replenishment.
Partial specialist failures remain visible; the plan is marked incomplete. No agent
can approve or execute an action through orchestration.

Guarded action techniques

Typed immutable payload + merchant scope + finite-state lifecycle + unique workflow
identity + atomic execution claim + fresh source/policy validation + transaction/savepoint
rollback + persistent audit. Pending → validated → awaiting approval → approved →
executing → executed/failed. Rejected/failed paths do not authorize execution.
Stale execution may retain approved state with a blocked outcome; check result/policy,
not only status. Repeated completed execution returns the prior result without another
price-history entry. SQLite tests do not prove PostgreSQL concurrent locking behavior.

Appendix B: API, database and security

API, data model and security reference

Actual routes

Except root routes, prefix is /api/v1. OpenAPI at /docs is the detailed schema reference.

Method	Path after prefix	Purpose
GET	/health; /health/detailed	Liveness; sanitized DB/optional Redis diagnostics
GET	/merchants	Available merchant summaries
GET	/products?merchant_id=ID	Paginated catalog, bounds and analytics
GET	/inventory?merchant_id=ID	Current catalog inventory view
GET	/recommendations?merchant_id=ID	Persisted saved recommendations
POST	/pricing/recommend	PricingRequest → PricingRecommendation
POST	/restock/recommend	RestockRequest → RestockRecommendation
POST	/promotion/recommend	PromotionRequest → PromotionRecommendation
POST	/listing/recommend	ListingRequest → ListingRecommendation
POST	/orchestrate	Merchant goal/product scope → advisory ActionPlan
POST / GET	/actions	Create typed guarded proposal / list merchant actions
GET	/actions/{id}?merchant_id=ID	Scoped action detail
POST	/actions/{id}/approve; /reject	Human decision; no execution
POST	/actions/{id}/execute	Separate confirmed execution/revalidation
GET	/action-history?merchant_id=ID	Persistent audit events
GET	/integrations/shopify/status; /last-sync	Connection and last synchronization summary; merchant_id query required
POST	/integrations/shopify/sync	Manual bounded read-only import

Root GET /health returns {status:ok}; root / returns app metadata.

Specialist requests include product_id and frontend-supplied merchant_id; legacy pricing/restock and other single-product contracts permit optional scope for compatibility.

Orchestrator requires merchant scope. Typed schemas reject invalid/unknown fields and bound quantities/limits. No endpoint executes a recommendation during analysis.

Action writes and all Shopify routes are development/test only, 403 in production.

There is no public reset, supplier-order, Shopify-write or AgentRun-history API.

Data model map

Entity	Main responsibility
Merchant	Store identity and ownership; not an authenticated login principal
Product	SKU/name/category/description, cost/selling price, status, local/Shopify source
Inventory	Physical/reserved/unavailable quantities and reorder thresholds; one product association
Order / OrderItem	Merchant-scoped sales and product-level quantities/prices
PriceHistory	Product price changes and change metadata
AgentRun	Run/status model exists; current session UI is not persisted through it
Recommendation	Structured agent proposal and optional saved history
Action	Immutable execution payload, status, workflow identity, policy/result and timestamps
Approval	Human decision records associated with actions

AuditLog	Persistent workflow events and safe before/after execution metadata
ShopifyConnection	Bound store/merchant identity, locations and sync status; no token storage
ExternalProductMapping	Shopify product/variant → local product identity, source/location metadata
ExternalOrderMapping	Shopify order → local order identity for idempotent sync

Foreign keys/relationships enforce ownership and associated-record integrity; SQLite foreign keys are enabled in tests/app. Unique external/workflow identities prevent accidental duplicates. Prices use Decimal/Numeric(10,2); timestamps are normalized UTC. Alembic has three current revisions, ending at 11shopify_read_only. Migrations and explicit demo seed are separate from the serving process. Read source models/migrations for exact fields rather than treating this summary as a complete schema export.

Security and guarded execution

Environment-managed backend credentials; VITE_ settings are public and contain no Shopify/DB secrets. Closed read-only Shopify GraphQL registry, no mutation dispatcher or redirects, bounded timeout/retries/pagination. Shopify remains the source of truth; local action results are not synchronized to it.

Scoped merchant-product/action/recommendation ownership checks protect accidental cross-store access, but there is no authenticated identity binding. CORS controls browser origin access, not merchant authentication. Legacy omitted scope and unauthenticated read APIs make trusted local/private access essential. Production keeps write routes blocked; do not weaken that gate for a demonstration.

Workflow: validated immutable proposal → policy checks → awaiting human approval → review decision → separate execution confirmation → atomic claim and fresh validation → local mutation/simulation → audit. Policies check current price/cost, margin, quantity, stockout/inventory context, source listing facts and approval state. Idempotency, transactions/savepoints, rollback, source comparisons and PostgreSQL locking mechanisms protect execution. Basic PostgreSQL execution passed; locking stress has not been measured.

Failure messages/logs are sanitized; debug is false. No credentials are persisted in integration mappings or returned to the frontend. Inventory gaps/unknown cost fail safely. Public production would require authentication/authorization, ingress rate controls, provider HTTPS/TLS/backups and broader concurrency verification. No compliance guarantee or audited enterprise security claim is made.

Appendix C: Real browser demonstration evidence

Figure 1. Merchant-scoped dashboard

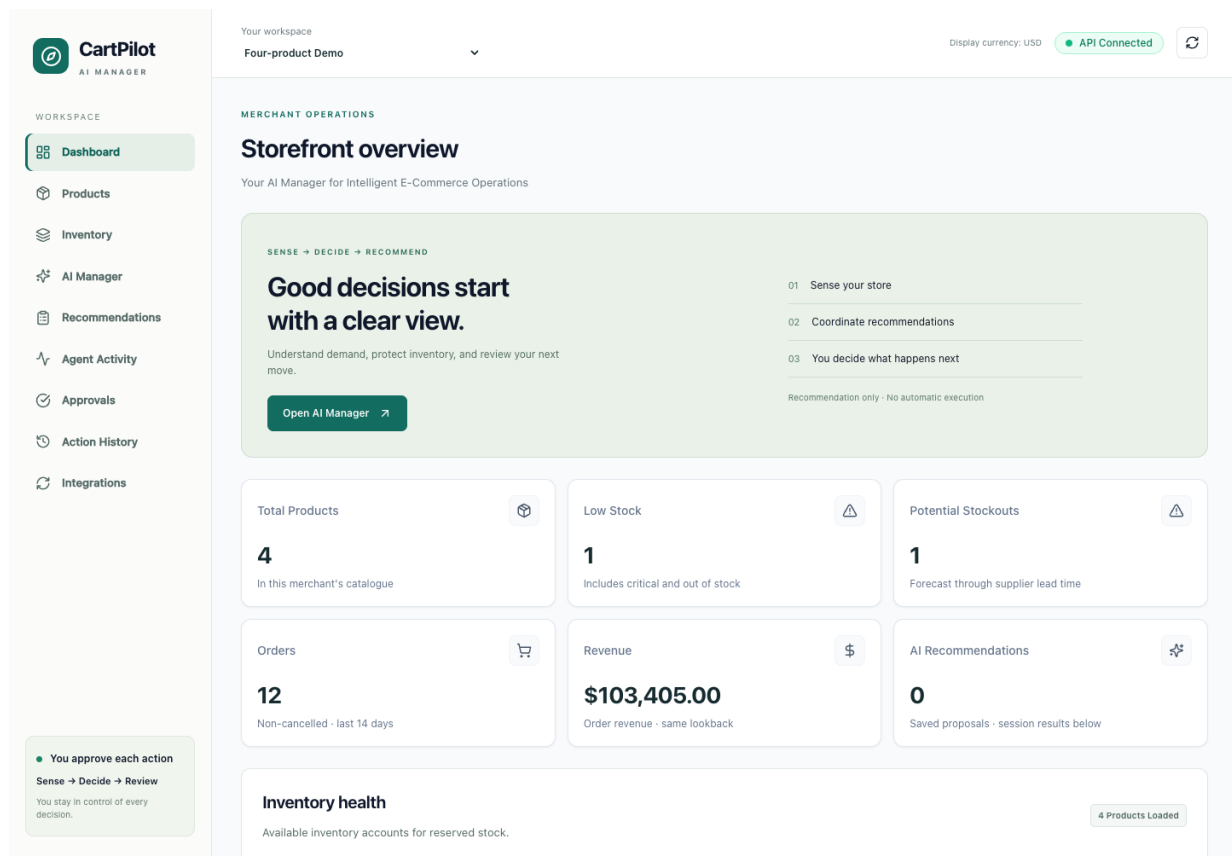


Figure 2. Mouse pricing recommendation; heuristic confidence

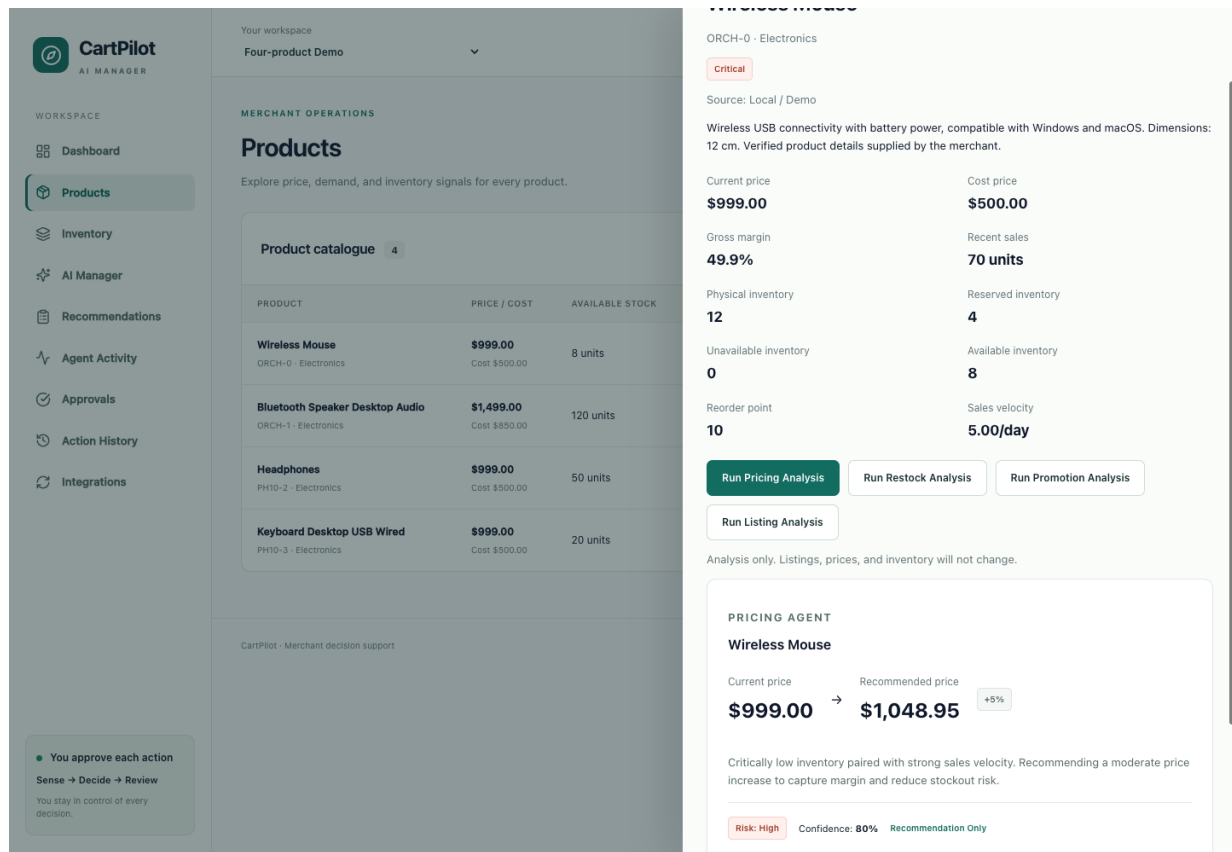


Figure 3. Restock demand and quantity assessment

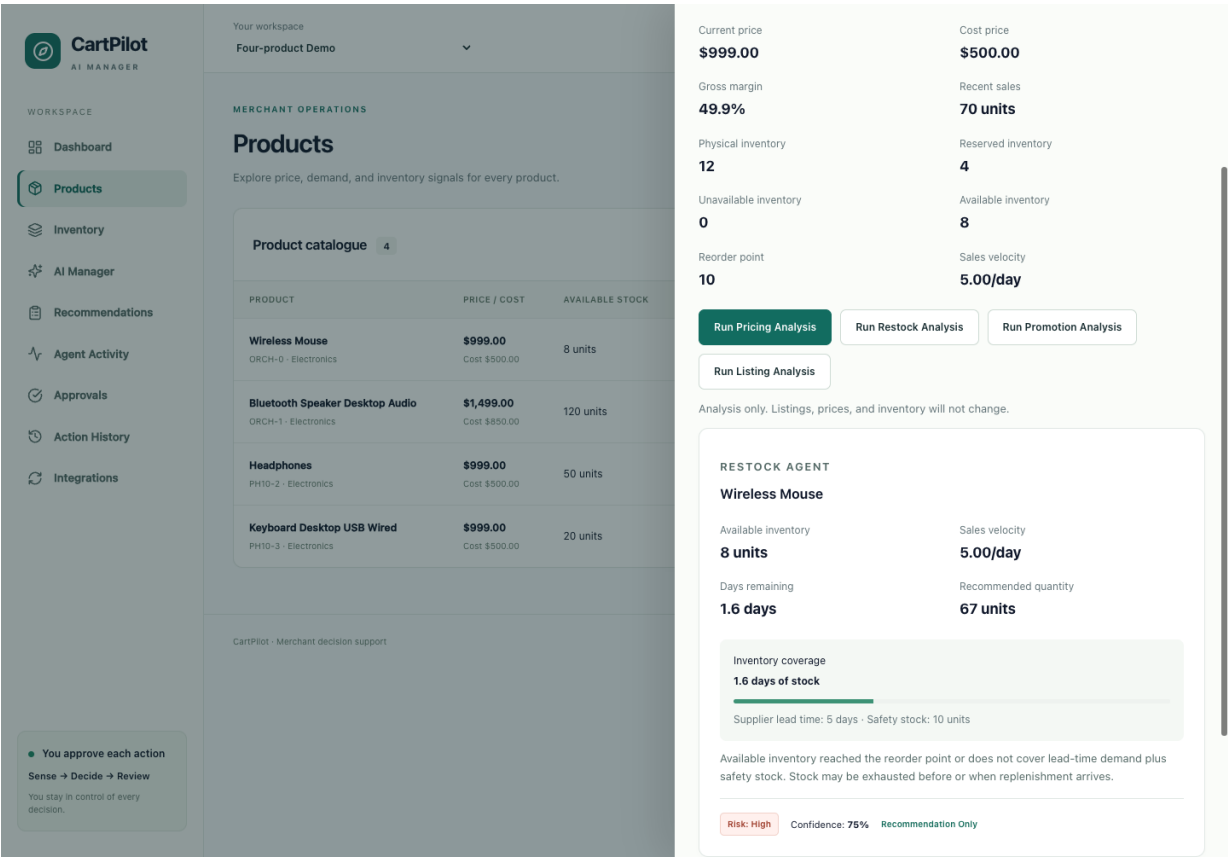


Figure 4. Inventory/margin-safe Speaker promotion

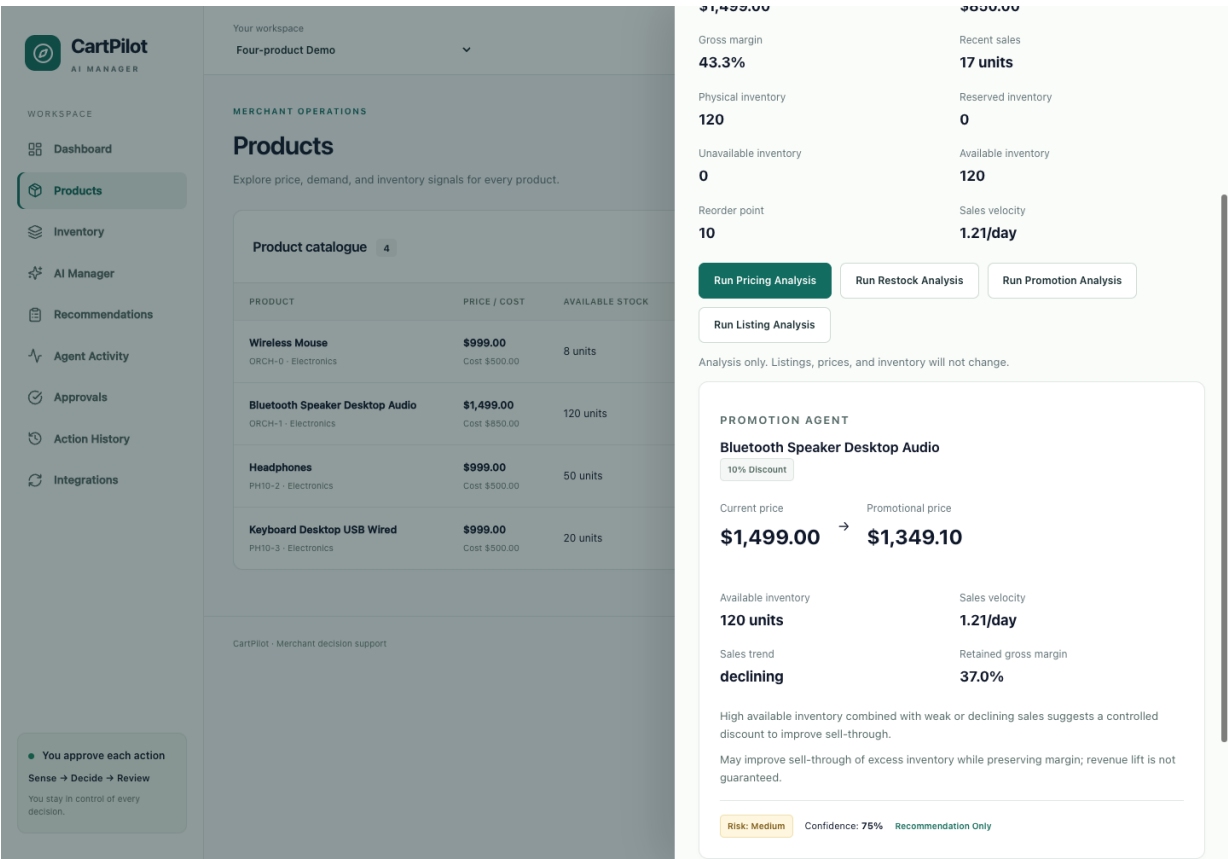
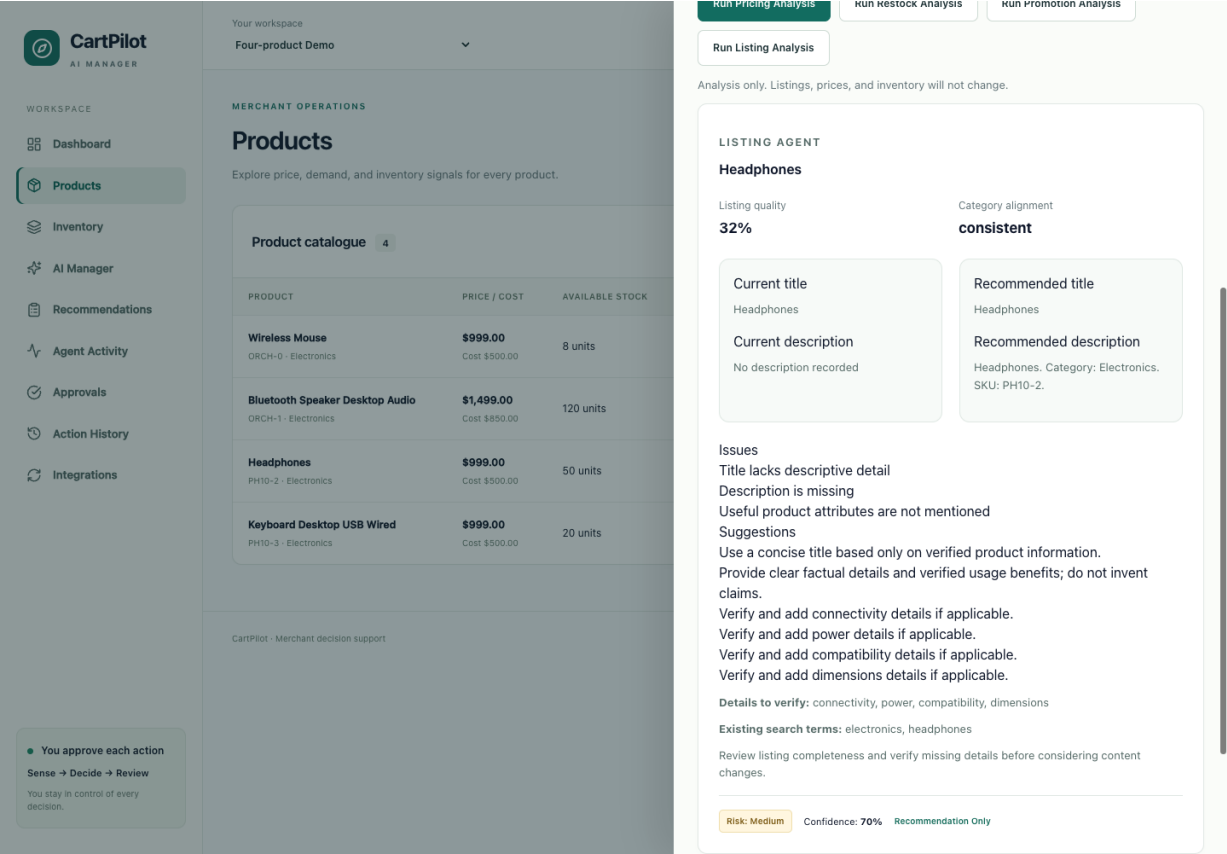


Figure 5. Grounded Headphones listing review



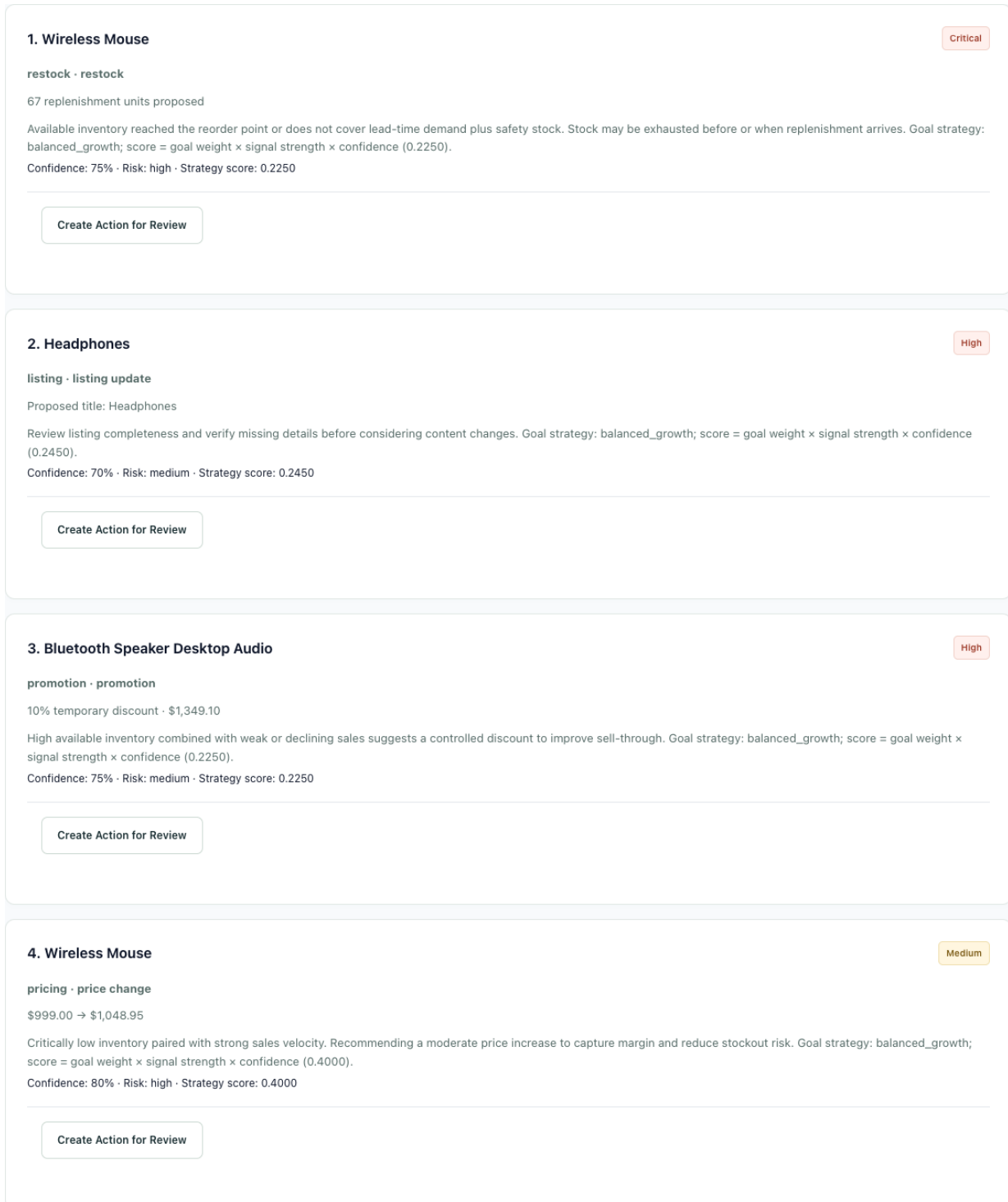


Figure 7. Explicit inventory/promotion conflict

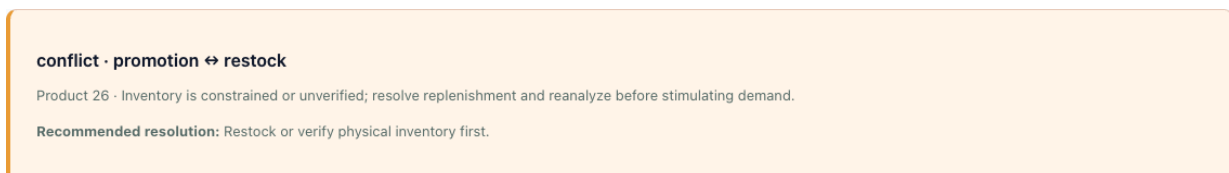


Figure 8. Separate execution confirmation

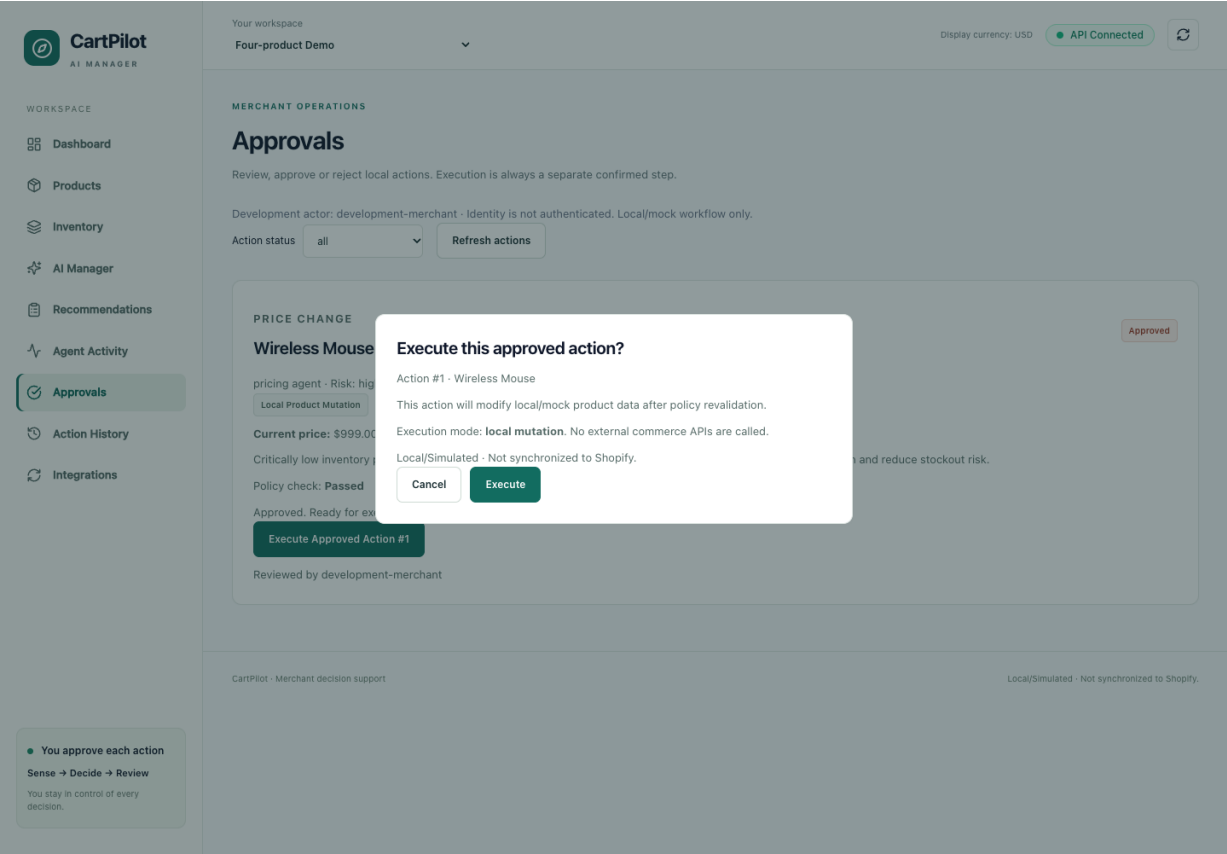


Figure 9. Persistent workflow audit

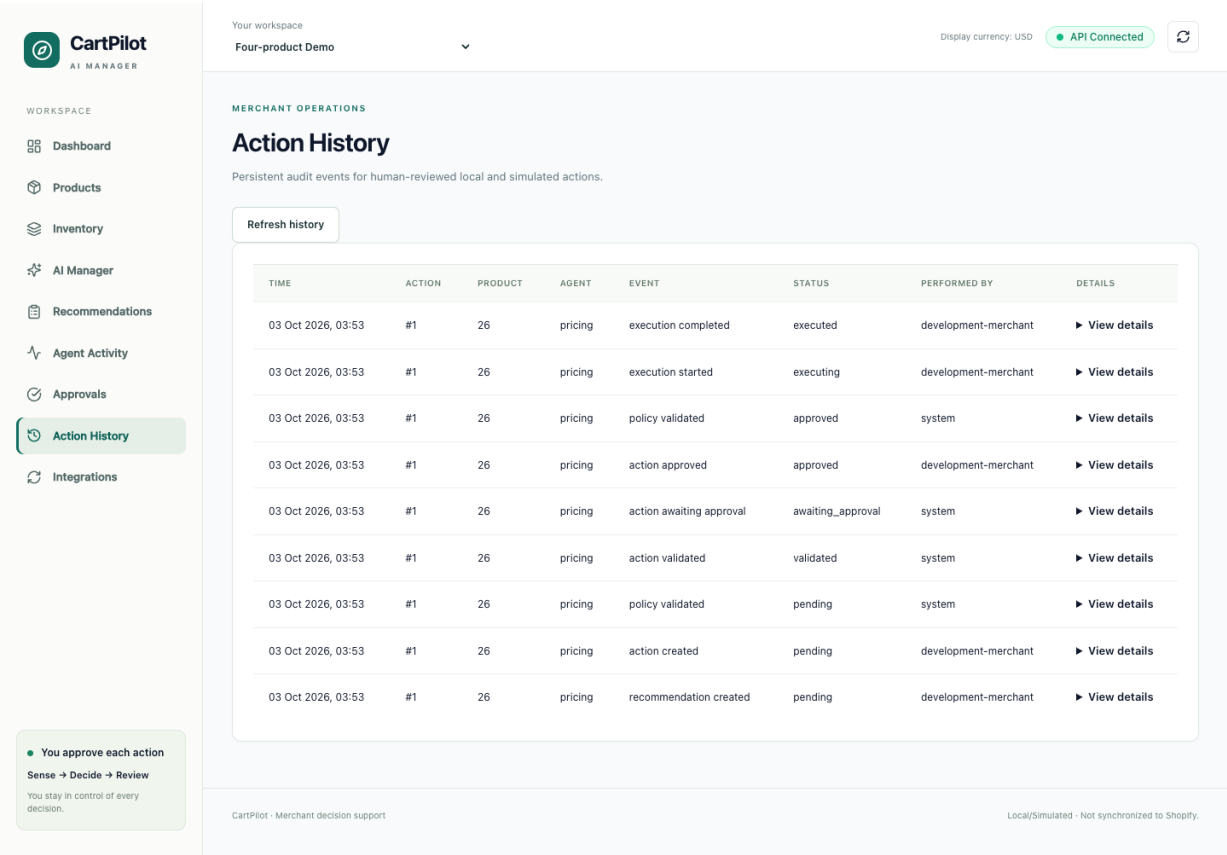


Figure 10. Unsafe movement policy block

WORKSPACE

Dashboard

Products

Inventory

AI Manager

Recommendations

Agent Activity

Approvals

Action History

Integrations

You approve each action

Sense → Decide → Review

You stay in control of every decision.

Your workspace

Four-product Demo

Display currency: USD

API Connected

MERCHANT OPERATIONS

Approvals

Review, approve or reject local actions. Execution is always a separate confirmed step.

Development actor: development-merchant - Identity is not authenticated. Local/mock workflow only.

Action status

all

Refresh actions

PRICE CHANGE

Wireless Mouse · Action #2

pricing agent - Risk: high - Human approval required

Local Product Mutation

Current price: \$1,048.95 → Proposed price: \$1,573.43

Critically low inventory paired with strong sales velocity. Recommending a moderate price increase to capture margin and reduce stockout risk.

Policy check: Blocked

Price change exceeds configured maximum change.

Execution Blocked

{
 "outcome": "blocked",
 "violations": [
 "Price change exceeds configured maximum change."
]
}

PRICE CHANGE

Wireless Mouse · Action #1

pricing agent - Risk: high - Human approval required

Local Product Mutation

Current price: \$999.00 → Proposed price: \$1,048.95

Critically low inventory paired with strong sales velocity. Recommending a moderate price increase to capture margin and reduce stockout risk.

Policy check: Passed

Reviewed by development-merchant

Executed Successfully

03 Oct 2026, 03:53 · Action #1

{
 "outcome": "executed",
 "mode": "local_mutation",
 "before": {
 "selling_price": "999.00"
 },
 "after": {
 "selling_price": "1048.95"
 }
}

CartPilot - Merchant decision support

Local/Simulated - Not synchronized to Shopify.

Figure 11. Mocked Shopify read-only import; not a live store

WORKSPACE

Dashboard

Products

Inventory

AI Manager

Recommendations

Agent Activity

Approvals

Action History

Integrations

You approve each action

Sense → Decide → Review

You stay in control of every decision.

Your workspace

Shopify Offline Demo

Display currency: USD

API Connected

STORE CONNECTIONS

Integrations

Import store data for analysis. Shopify remains the source of truth.

Shopify

READ ONLY

Connected

Connected. Shopify is the source; CartPilot is a local analysis cache.

Store: fixture-store.myshopify.com

API version: 2026-10 · Currency: USD

Credentials are configured in the backend environment. The access token is never shown here. Local/demo data works independently.

Test Connection

Sync Now

Synchronization completed

Last Sync: 03 Oct 2026, 03:53

Products: 3 · Inventory records: 3 · Orders: 1 · Skipped orders: 0 · Errors: 0

Local/Simulated actions are not synchronized to Shopify. Sync imports Shopify values into the cache.

CartPilot · Merchant decision support

Local/Simulated · Not synchronized to Shopify.

CartPilot • Verified MVP • Academic details pending

18